



TigerEye

Cycle 2 Report

Authors:

Amelia Snyder

Mihir Patel

Amy Prieto

Andrea McCarroll

Nicholas McGallagher

Aidan Anderson

Comp 4710 Senior Design Team #28

Department of Computer Science and Software Engineering

Samuel Ginn College of Engineering, Auburn University

Table of Contents

1 System Metaphor (Andrea).....	2
2 Cycle Intent (Andrea, Amy).....	2
3 Management Plan & Project Management (Aidan, Mihir, Andrea)	2
1. Semester Assignments	3
2. Current Assignments	4
3. Development Schedule	4
4. Meeting Minutes & Cadence.....	5
4 User Stories (Mihir).....	5
5 Design Documentation (Mihir, Nick & Amelia)	6
1. Overall Product Diagram	6
2. Parser Engine.....	6
3. Alert Engine	7
a. Alert Engine Update – Cycle 2 (Aidan)	9
4. Graphic Engine (Andrea)	9
5. Elasticsearch Database Implementation (Aidan & Mihir)	11
1. OTX Source Curation (Amy)	12
6. Domain Model (Nick)	12
7. Core Entities (Nouns).....	13
8. Domain Relationships (Logic)	14
9. Alternative Domain Model.....	14
10. Alternate Core Entities (Nouns).....	14
11. Alternate Domain Relationships (Logic)	14
6 Risk Mitigation (Aidan)	16
7 Lessons / Challenges (Amy & Mihir)	17
8 Sponsor Approval (Amy).....	19
9 Memoranda (Nick).....	19
January 28, 2026 – TigerEye Meeting Notes	19
February 11, 2026 – TigerEye Meeting Notes.....	21
February 18, 2026 – TigerEye Meeting Notes.....	22

1 System Metaphor (Andrea)

System metaphor: “A Bird’s Eye in the Sky”

TigerEye can be described to stakeholders as a “Bird’s Eye in the Sky” — a centralized vantage point that rises above individual systems to provide comprehensive awareness of the threat landscape. Like a bird flying high above the ground, TigerEye gives security teams a broad, unobstructed view of their environment instead of limiting them to isolated alerts or disconnected data sources.

From this elevated perspective, TigerEye aggregates network activity and threat intelligence into a single, unified platform. This allows analysts to see patterns, correlations, and anomalies that would be difficult to recognize from a ground-level view. The parser engine acts as the bird’s vision by collecting and translating raw network traffic, threat intelligence provides knowledge of known dangers, and the alert engine serves as the warning signal when suspicious activity is detected.

This higher-level visibility improves situational awareness and enables faster, more confident decision-making. By seeing the full picture, security teams can prioritize threats, detect risks earlier, and respond more effectively. Ultimately, TigerEye transforms fragmented information into clear, actionable insight, allowing defenders to move from reactive responses to proactive security.

2 Cycle Intent (Andrea, Amy)

Cycle takeaways:

In cycle 2, we continued to advance our primary objective of building a complete pipeline from network traffic ingestion to structured output. Our focus included storing parser engine results within an Elasticsearch database to support downstream processing and visualization. In parallel, we expanded our work with Open Threat Exchange (OTX), which remains a key focus area, by researching and curating relevant pulses to build out our threat intelligence dataset. Additionally, we established a framework for consistently feeding processed data into Grafana, enabling the development of our user interface and visualization capabilities. Throughout this cycle, we continued to divide responsibilities across the team to support parallel progress on core system components.

3 Management Plan & Project Management (Aidan, Mihir, Andrea)

1. Semester Assignments

- a. Architect – Aidan Anderson

- i. Helping with engineering tasks across different project sectors
 - ii. Designing overall application capabilities based on customer's desired MVP
 - iii. Establishing initial data pipelines from McCrary's testbed environment.
 - iv. Integrating our varied sectors with each other to accomplish various objectives and tasks
- b. Backend Engineer - Amelia Snyder
 - i. Developing the initial parser engine to interpret network traffic
 - ii. Researching and implementing best export format to deliver data to our pattern engine/system
 - iii. Formatting IT/OT network traffic, in a uniform manner.
- c. API integration/Backend Engineer - Mihir Patel
 - i. Researching available sources of OSINT related to both IT and ICS environments
 - ii. Designing a pattern engine to recognize potential threats based on OSINT sources (API integration)
 - iii. Selecting and implementing database solutions for our overall application
- d. Full Stack Engineer - Nick M.
 - i. Working alongside GUI Engineer to select best visualization tools, languages, and products
 - ii. Ensuring accurate data is integrated into threat cases
 - iii. Tagging data with timestamps to help visualize threat timelines for operators
- e. Graphical Interface Engineer - Andrea McCarroll
 - i. Designing the UX/UI for our operators and analyst in mind
 - ii. Researching pre-built infrastructure that could be leveraged to reduce dev time
 - iii. Creating customized dashboards to present our key features such as event timelines, traffic flow diagrams, and incident case queues
- f. Open-Source Engineer - Amy Prieto
 - i. Providing feedback & instruction on architectural decisions to ensure containerization & open-source capability
 - ii. Managing the GitHub code repository
 - iii. Collaborating in a full-stack capacity, with back end and front-end engineers on key feature capabilities

- iv. Researching OT cyber domain tools, providing direction on key feature development, and ensuring value to the overall market.

2. Current Assignments

- a. Aidan + Nick - Alert Engine
 - i. Write Python code to generate structured alert files based on Indicators of Compromise (IoCs)
 - ii. Pull pulse data from Open Threat Exchange (OTX) and extract relevant IoCs, primarily IP addresses
 - iii. Compare OTX-derived IoCs against parser-generated or simulated network data
 - iv. Generate structured JSON alert output formatted for future Elasticsearch ingestion
 - v. Establish a functional pipeline from OSINT ingestion to alert generation
- b. Mihir + Amelia - Parser Engine
 - i. Confirm ability to parse DNP3 traffic using the provided test case in the repository
 - ii. Ensure parser exports clean, and has structured JSON output
 - iii. Move JSON output into VM environment running Elasticsearch
 - iv. Validate successful ingestion of parser output into Elasticsearch
- c. Amy + Andrea - Threat Intelligence + UI Concept
 - i. Research and compile relevant OTX pulse IDs related to industrial environments and DNP3 traffic
 - ii. Deliver a curated list of usable pulses for the Alert Engine team
 - iii. Develop a GUI mockup for inclusion in the upcoming report
 - iv. Design visual concepts such as alert timeline, case view, and IoC match visualizations
- d. Everyone
 - i. Study foundational Threat Intelligence concepts
 - ii. Understand the distinction between IoCs and CVEs
 - iii. Review and become familiar with the Pyramid of Pain to support engine design decisions

3. Development Schedule

- Parser Engine Beta development: February 6th – February 20th
- OSINT API Beta development: February 6th – February 14th
- Task Check-in: February 11th
- Sponsor Check-in: February 19th or 20th (Based on Sponsor's Availability)
- Stretch goal: Live data pipeline established by February 22nd.
- Retrospective & Schedule assessment: March 4th

4. Meeting Minutes & Cadence

- All meetings begin with a retrospective of last week's assigned tasks
- Each member reports on task progression, resource needs, and if they're facing any blockers
- At the end of the meeting, each member tells the team their "sweat level" for the upcoming week, so adjustments can be made based on our joint bandwidth.

Every Wednesday – 6-7 PM	Every Thursday – 6-7 PM
Required weekly meeting to discuss progress and go over tasks to be completed before the next meeting.	Optional workshops to follow up technical obstacles and collaborate on solutions.

4 User Stories (Mihir)

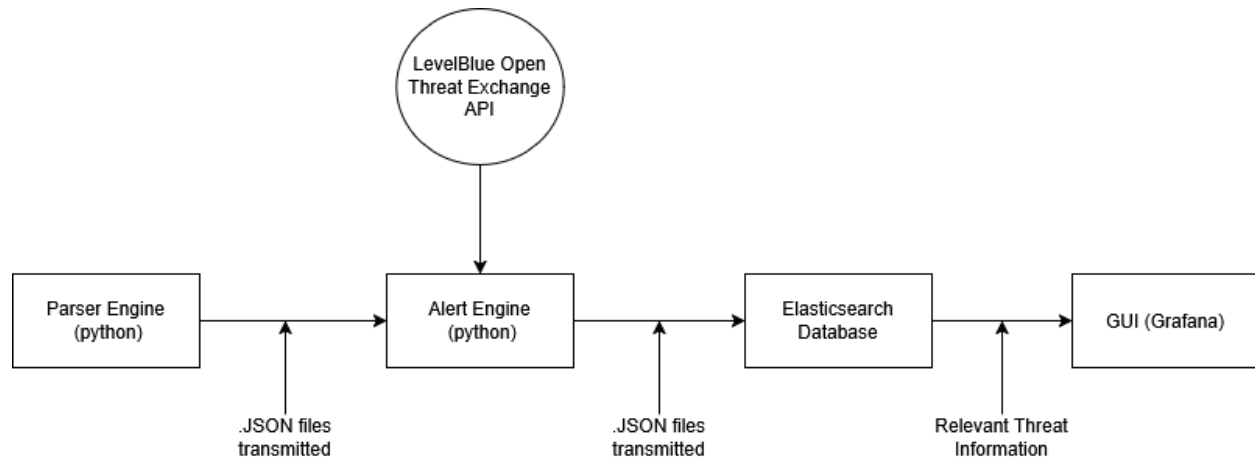
- As a security operator, I want the ability to view alerted threats, on a specific device, based on open-source intelligence (OSINT), so that I can better assess defensive countermeasures.
- As a SOC analyst, I want unified IT/OT alerts in a single timeline, so that I can identify cross-domain attack patterns.
- As a cyber security professional and system user, I want to have the ability to set up the system to prioritize alerts and events that my company had determined are our Tier 1 level threats, so that I can better customize the system dashboard to fit my needs.
- As a SOC analyst, I want an interactive, real-time baseline of IT/OT behavior with correlated security events, so that I can identify meaningful deviations, understand incident context, and reduce false positives in industrial environments.

These user stories were collaboratively developed to reflect the core workflow and learning objectives of the TigerEye platform, with each story aligned to support effective threat hunting and situational awareness across IT and OT environments. Together, they emphasize visibility into OSINT-driven device alerts, correlation of IT and OT events within a unified timeline, and the ability to prioritize critical threats based on organizational risk. The final story connects these capabilities through real-time behavioral baselining and interactive visualization, ensuring alerts are contextualized, actionable, and meaningful. Collectively, the user stories define a cohesive workflow that moves from data ingestion and intelligence to correlation, prioritization, and analyst-driven insight. For the Cycle 1

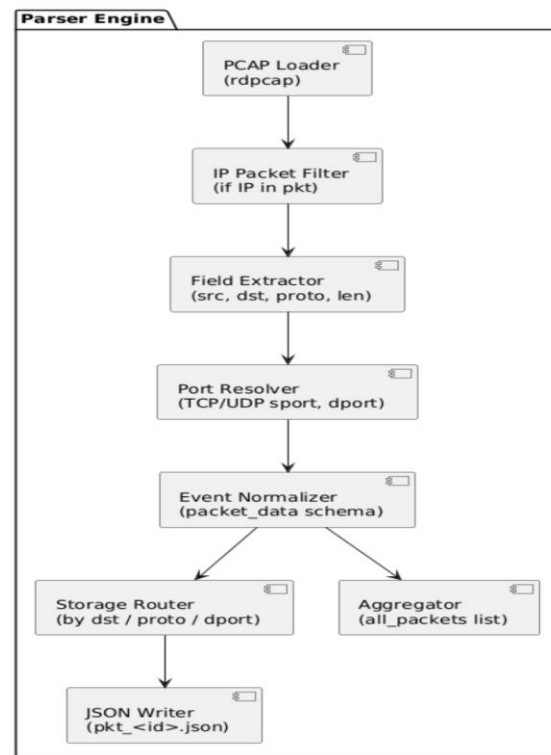
report, our use cases remained the same. However, we thought about each user and how they would use a final product more in depth since our actual development had begun.

5 Design Documentation (Mihir, Nick & Amelia)

1. Overall Product Diagram



2. Parser Engine

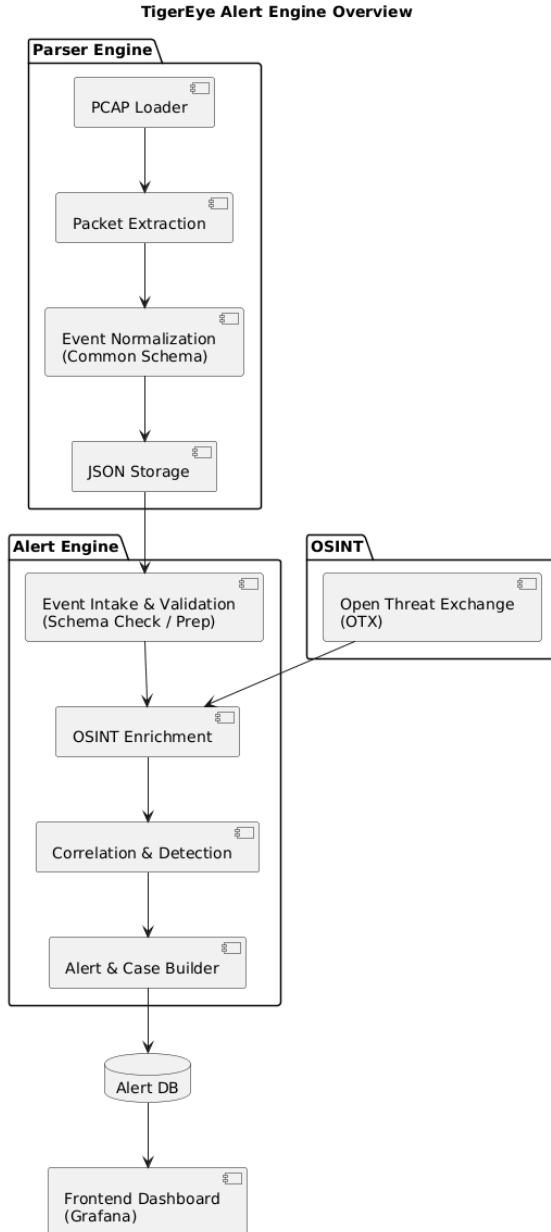


The parser engine's main purpose is to read in .pcap files, sort through their data and collect the relevant information based on our alert engine, save the .pcap logs as .json files, and upload them to a database that can be accessed by the alert engine. To accomplish this, we collected a small sample of .pcap files to test our engine. First, we loaded the pcap files, then created a basic parse program to read through the files. This program paid attention to several data points from the logs, including the timestamp, the source IP, the destination IP, and the protocols used. It also includes data logs about what the source and destination ports the traffic was using. This information was stored in a dictionary variable, which was later saved in a folder scheme as .json files.

a. Parser Engine Update – Cycle 2 (Amelia)

At this moment, the parser engine is accessible by the elastic search database, and it can be used to provide data metrics on the .json files collected. The parser engine itself has not gone through significant changes, as the main goal with the parser in this cycle was to begin to connect it to the larger system. Moving forward, we will be creating a secondary version of the parser engine without the nested .json file scheme. With the nested file structure, .json files are separated by their destination IP, destination ports, and the protocol used. This secondary version will be a flat storage structure, where all .json files are stored in the same folder. We will be testing how both versions interact with the larger data pipeline and the elastic search database.

3. Alert Engine



The Alert Engine will begin once structured events are received from the Parser Engine. Its first stage, Event Intake & Validation, will ensure that incoming events conform to the expected schema, verifying required fields while preparing the data for downstream processing. From there, events will move into OSINT Enrichment, where relevant indicators such as IP addresses, domains, or ports will be queried against external threat intelligence sources, specifically Open Threat Exchange (OTX). In the current development cycle, the primary focus will be retrieving IP address intelligence from OTX and comparing it to the IP addresses extracted by the parser. Any returned intelligence—such as known

malicious associations or related threat pulses—will be attached to the event to increase contextual awareness.

In a later sprint, the now enriched events will pass into a Correlation and Detection phase, where rule-based logic and pattern analysis will be applied to identify suspicious events. This stage will determine whether an event or series of events meets predefined detection criteria. When a detection threshold is met, the Alert & Case Builder will construct a structured alert, assigning a level of severity and confidence. These alerts and cases will then be stored in an Alert Database, where they will be accessible to the frontend dashboard for SOC analysts to review and prioritize.

a. Alert Engine Update – Cycle 2 (Aidan)

Currently, we have developed a 50-line prototype that can extract IPv4 & IPv6, our primary indicator of compromise, from Open threat exchange and compare it to an input list of IPs. If any of the input IPs are found in the OTX-gathered malicious IPs, then they are flagged. All flag IPs are put into a set, that is returned to the user to test the engine's logical integrity. Overall, what makes this a prototype is the lack of integration with the parser json. Integrating these two engines in a productional level manner will be integral to our graphical interface and tool's value proposition to operators.

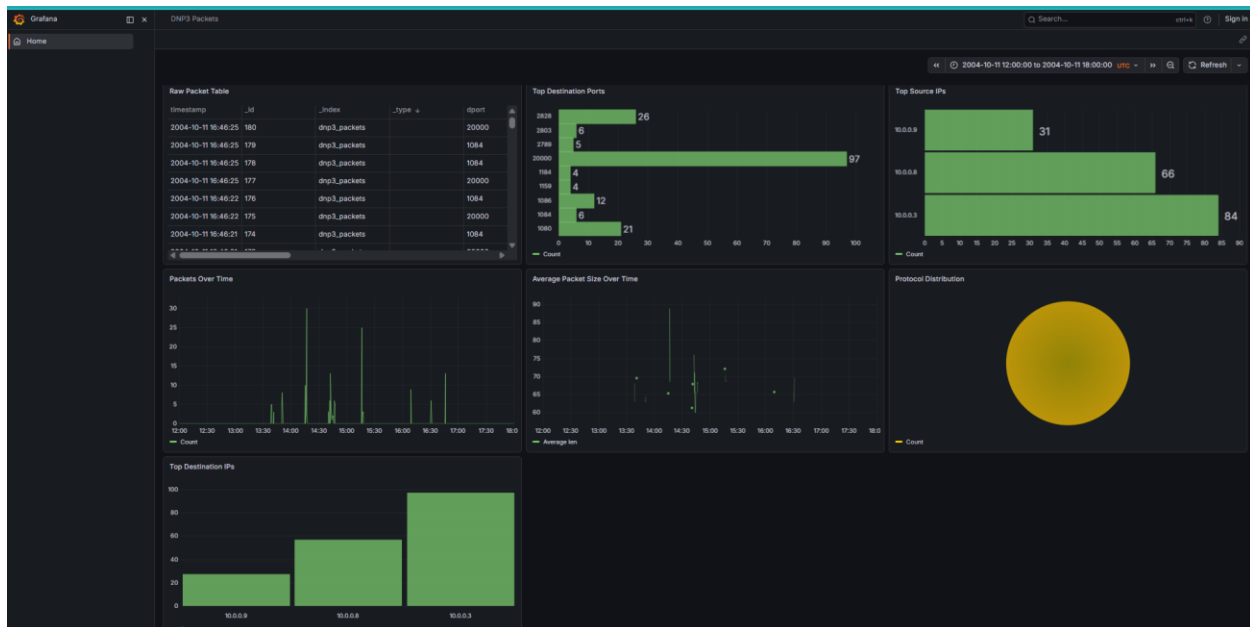
Which leads me to the next steps; in our upcoming cycle, the goal is to iterate through the key-value JSONs generated by the parser engine and compare it to the malicious IPs being aggregated. In addition, the curated pulse list, in development, will be used to better direct the engine's collection and surveillance. A benefit of our elasticsearch database is designed to use unique IDs to optimize order transversals, which makes up for the bandwidth strain of high volume returns from OTX. This way, we take a more streamlined approach to our pipeline, while also tracking which threats did or did not appear.

4. Graphic Engine (Andrea)

At the current stage of development, the Graphic Engine is in a prototype/testing phase. The image below is proof that we were able to create a functional Grafana dashboard and successfully integrated with parsed network data. The prototype dashboard visualizes DNP3 traffic and includes several key panels:

- i. **Raw Packet Table:** Displays parsed network events with fields such as timestamps, source/destination ports, and protocol type.
- ii. **Top Source and Destination IPs:** Highlights the most active devices in the network, allowing analysts to quickly identify dominant communicators.

- iii. **Top Destination Ports:** Provides insight into commonly used ports, helping detect abnormal service usage.
- iv. **Packets Over Time:** Visualizes traffic burst and temporal patterns, supporting behavioral analysis.
- v. **Average Packet Size:** Helps identify anomalies in payload size that may indicate unusual activity.
- vi. **Protocol Distribution:** Confirms protocol usage within the environment (currently focused on DNP3 traffic).



The next phase of development will focus on evolving the Graphic Engine into a more meaningful, analyst-friendly format that supports real cyber security workflows rather than simple data visualization. Planned enhancements include:

- i. **Alert-Centric Dashboards**
 - a. Display alerts generated by the alert engine instead of raw traffic
 - b. Include severity levels, confidence scores, and IoC matches
- ii. **Case & Timeline Visualization**
 - a. Interactive timelines showing sequences of related events
 - b. Case-based views that group alerts for investigations
- iii. **Drill-Down Capabilities**
 - a. Ability to click on alerts, IPs, or devices to view deeper context
 - b. Expand from high-level summaries to packet-level details
- iv. **Threat-Intelligence Enrichment**
 - a. Integration of OTX data directly into visualizations

- b. Highlight known malicious IPs or indicators in real time
- v. **Behavioral Context**
 - a. Visual baselines of normal activity
 - b. Identification of deviations and anomalies
- vi. **User Customization**
 - a. Analyst-defined filters and priority views (e.g., Tier 1 threats)
 - b. Custom dashboards tailored to organizational needs

The implementation of these enhancements is somewhat feasible within a one-month timeframe given the capabilities of Grafana and the current progress of the system. Several features are highly achievable within this period, including creating alert-based dashboards using structured JSON data, adding filters and visual refinements, implementing basic drill-down interactions, displaying IoC matches and enriched fields from OTX, and designing timeline visualizations through Grafana panels. Some aspects are only moderately feasible and depend heavily on backend readiness, such as full integration with the alert engine pipeline, consistent ingestion of enriched data from Elasticsearch, and maintaining stable data flow between Docker and VM environments. More advanced capabilities – such as behavioral baselining, anomaly detection visualization, fully developed case management workflows, and complex correlation visualizations requiring custom backend logic – are less likely to be completed within a one-month timeframe.

5. Elasticsearch Database Implementation (Aidan & Mihir)

Understanding the importance of proper data storage, access control, and scalability, the team agreed to use an elastic search database. This critical piece of architecture is used to anchor our data pipeline. All our generated alerts, trivial or critical, are stored here, while also being lightweight. The technical specifications of the device are 6GB of RAM with 100GB of Storage available,

The method we used to install this database is to virtualize a Debian 12 Linux VM using VMware Vcenter. Once, we set up the host and placed it on the network. We went through the basics of installing Elasticsearch 5.1.2 on to the host. Additionally, we enable OpenSSH as an available service with controlled keys for designated admins on our team, such as Aidan Anderson and Mihir Patel. We have run some tests of moving our parsed pcap traffic into, but we ran into networking issues. In cycle 2, we will troubleshoot access control standards to repair this issue, but the solution in the interim is using a docker container to store the data locally.

1. Cycle 2: Update

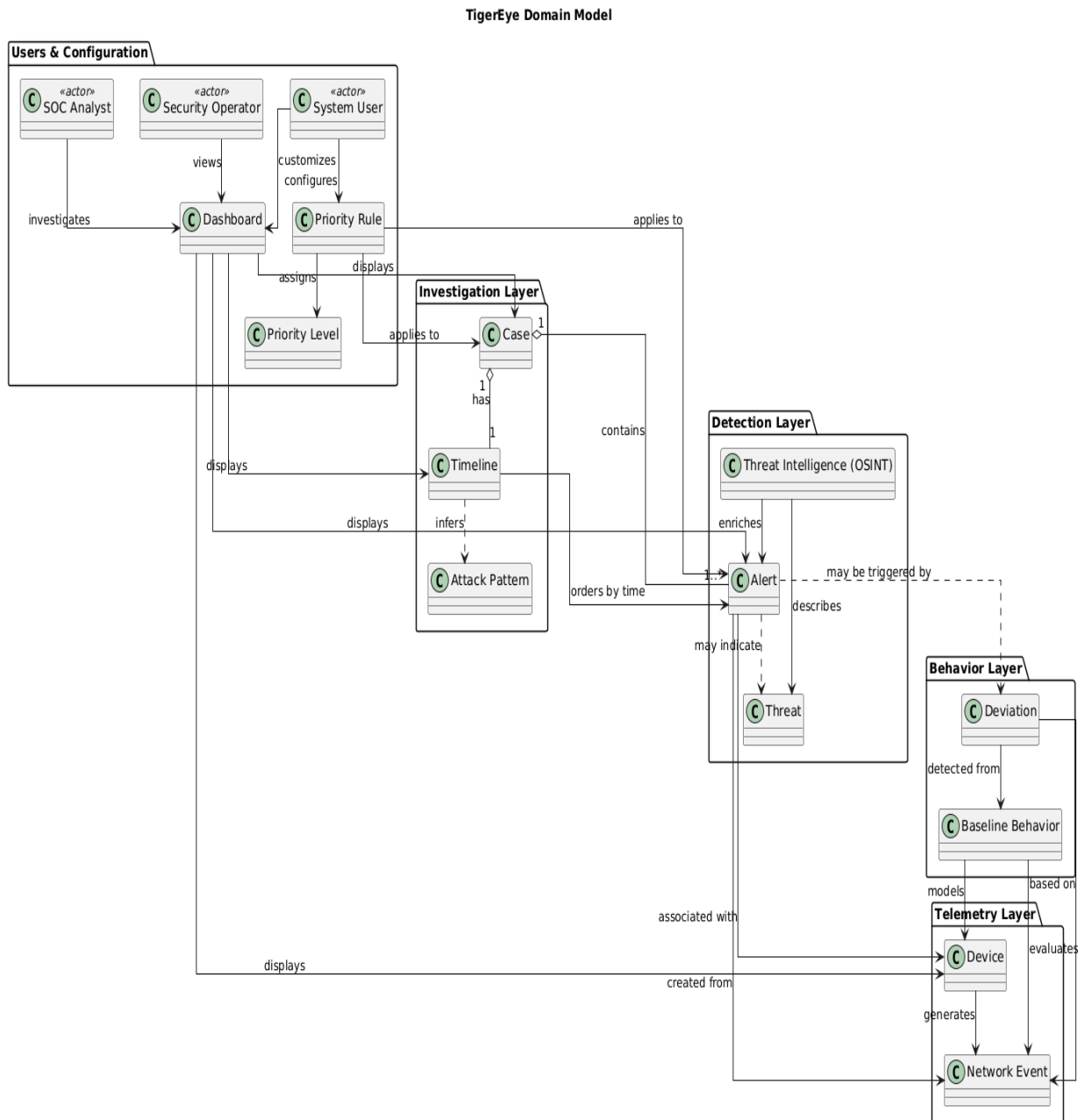
To store the enriched .json files that the current parser engine is outputting, we needed to create docker containers that run a local instance of the elastic search database. The way we accomplished this was by creating a docker compose yaml file that holds the specific information for the elastic search database such as the image type, container name, ports, volumes, and environment variables. This allows for quick customization of the database to determine the best local instance for our use cases. The yaml file also has a Grafana container that is created, so that we can make sure that data is being pulled properly to a frontend service. The containers are managed in docker desktop, so it is easy to turn the container on or off for quick development and testing.

For our upcoming cycle, we can hopefully get access to the virtual machine and properly set up our elastic search database. The goal is to get Aidan Anderson and Mihir Patel access to the VM to configure it, make sure data is being pulled from the combined parser & alert engine into the database, and make sure our Grafana dashboard is accessible to the rest of the team.

6. OTX Source Curation

In this Cycle we began our OTX Source Curation. This means we began selecting, validating and maintaining high quality threat feeds inside of OTX. Intelligence comes in the form of pulse. So IOCS like IP addresses, domains, or hashes. So, I have gone through and chosen reliable contributors/sources. This will help filter our low quality or noisy indicators, giving our environment. I am hand picking pulses that would best suit our traffic inflow and help us bring forward the most important parts of our data.

7. Domain Model (Nick)



8. Core Entities (Nouns)

- **Device:** An endpoint (IT/OT) within the Expo environment that generates network events.
- **Network Event:** Raw data (Suricata/SEL logs) captured from a Device.

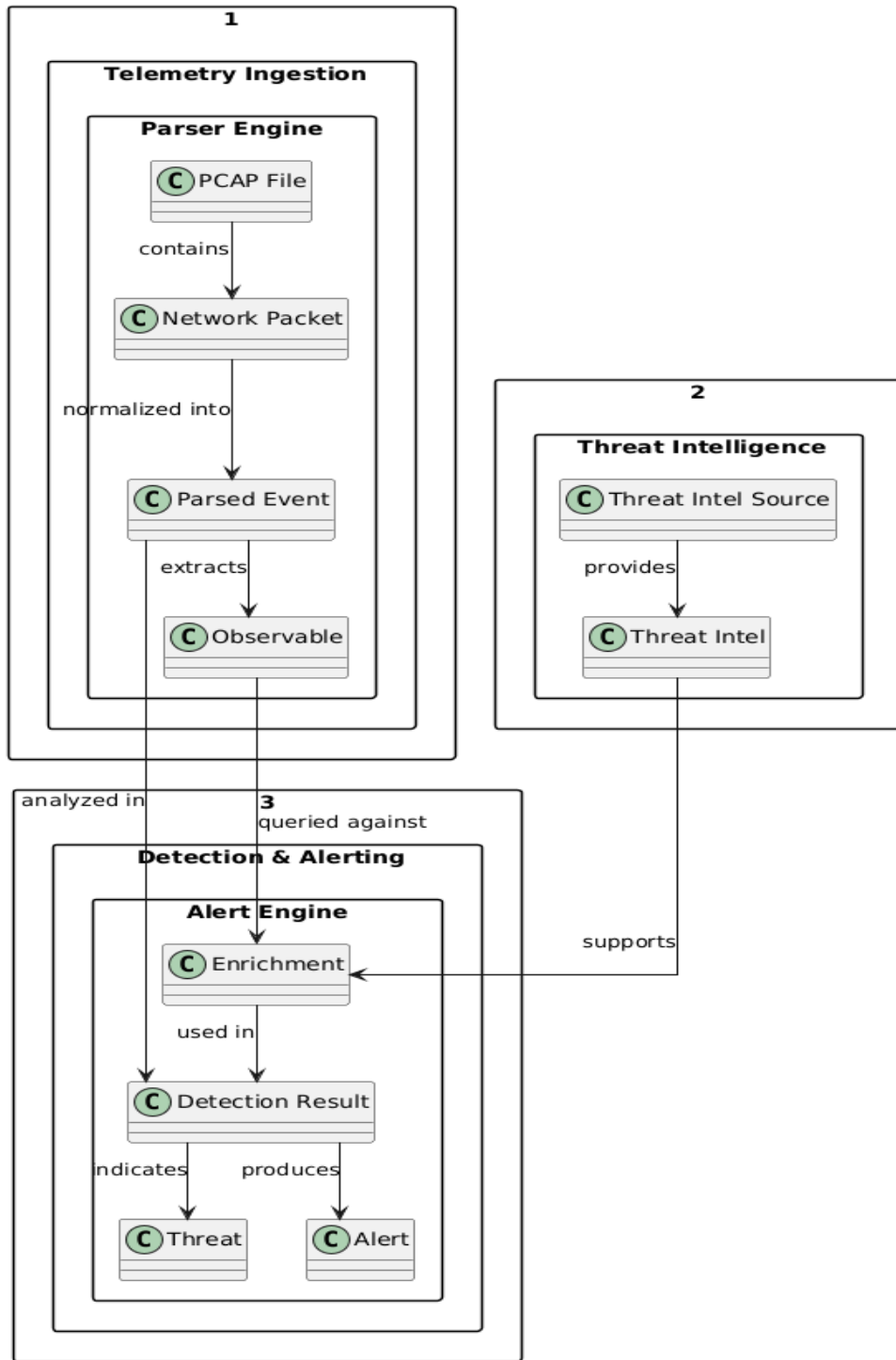
- **Alert:** A security event triggered when a Network Event matches a known threat pattern.
- **Threat Intelligence (OSINT):** External data from OTX used to "enrich" an Alert with context.
- **Case:** A logical grouping of related Alerts assigned for investigation.
- **Timeline:** A chronological sequence of Network Events and Alerts used to infer attack patterns.
- **Priority Rule:** User-defined logic used to assign a **Priority Level** (e.g., Tier 1) to specific threats.

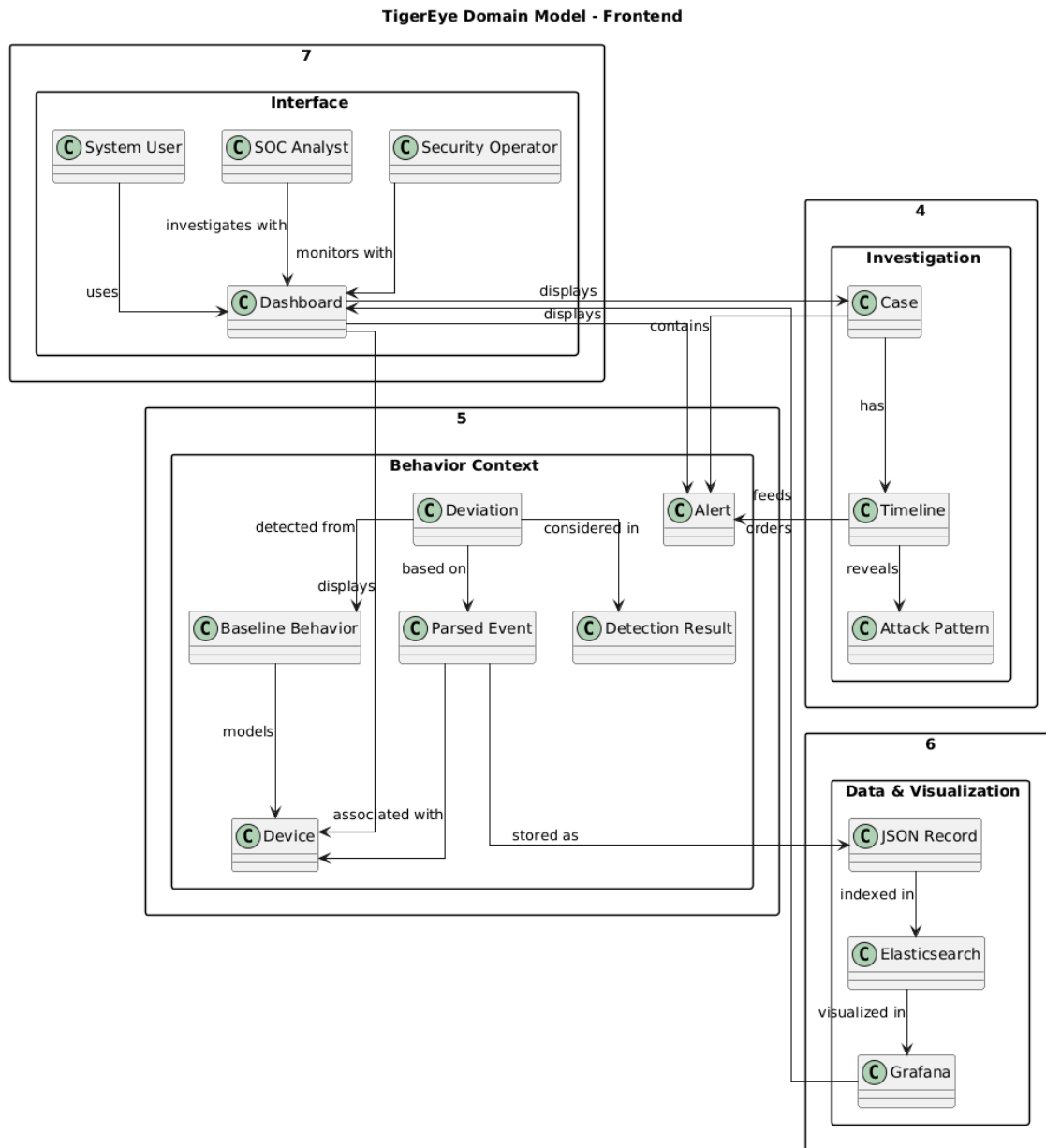
9. Domain Relationships (Logic)

1. **Telemetry Layer:** A **Device** generates multiple **Network Events**.
2. **Detection Layer:** A **Network Event** is evaluated to create an **Alert**, which is then "enriched" by **Threat Intelligence**.
3. **Behavior Layer:** **Network Events** are measured against **Baseline Behavior** to detect **Deviations**.
4. **Investigation Layer:** Multiple **Alerts** are aggregated into a **Case**, which has a corresponding **Timeline** to help analysts infer an **Attack Pattern**.
5. **Configuration Layer:** A **System User** defines **Priority Rules** that assign **Priority Levels** to specific **Dashboards** and **Cases**.

10. Alternative Domain Model (Nick)

TigerEye Domain Model - Core Pipeline (Parser + Alert Engine)





This version of the TigerEye domain model shows the system as a full investigation pipeline instead of just a collection of technical parts. It starts with the telemetry ingestion, where raw packet capture data is turned into structured parsed events and observables. Then it moves into threat intelligence and detection, where those observables are checked against outside intelligence and turned into detection results, threats, and alerts. After that, the model shows how alerts are organized into cases and timelines for investigation, while also using behavior context like baseline behavior and deviations to strengthen the analysis. Finally, it shows how the data is stored and visualized through Elasticsearch and

Grafana, and how users interact with everything through the dashboard. Overall, this version better shows how TigerEye works as a complete system from raw network data all the way to investigation and user interaction.

We changed to this version because the alert engine makes more sense logically and technically after the parser engine, not beside it or before it. The parser engine's job is to take raw PCAP data and turn it into structured parsed events and observables that the rest of the system can actually use. Until that parsing step happens, the alert engine does not yet have clean, normalized data to work with. By moving the alert engine after the parser, the system becomes more efficient and better organized, since detection logic can now operate on already extracted observables instead of repeatedly dealing with raw packet-level data. This also supports the code changes we are planning to make. Instead of having alert logic make repeated or loosely connected calls while parsing is still happening, the parser will first complete its work, produce structured outputs, and then pass those outputs into the alert stage. That means the code will shift toward a clearer pipeline: parse first, extract observables, enrich against threat intelligence, run detection logic, and then generate alerts. In practice, this should reduce redundancy, improve maintainability, and make the overall system easier to scale and debug. It also creates a cleaner separation of responsibility in the code, where the parser focuses on data transformation and the alert engine focuses on security analysis.

11. *Alternate Core Entities (Nouns)*

- **PCAP File:** A captured dataset of raw network traffic used as the initial input into the system.
- **Network Packet:** An individual unit of data within a PCAP file representing network communication.
- **Parsed Event:** A structured representation of network data created by the parser engine from raw packets.
- **Observable:** A specific data point (e.g., IP address) extracted from a Parsed Event for analysis.
- **Threat Intelligence:** External data (e.g., OTX) used to provide context about known malicious indicators.
- **Enrichment:** The process of comparing Observables against Threat Intelligence to add context.

- **Detection Result:** The outcome of analyzing Parsed Events and Enrichment data to determine potential threats.
- **Alert:** A security notification generated when a Detection Result indicates suspicious or malicious activity.
- **Threat:** A potential or identified security risk derived from Detection Results.
- **Device:** An endpoint (IT/OT) associated with network activity and events.
- **Baseline Behavior:** A model of normal activity for a Device used for comparison.
- **Deviation:** An abnormal behavior identified when activity differs from the baseline.
- **Case:** A grouping of related Alerts for investigation.
- **Timeline:** A chronological sequence of Alerts used to understand an incident.
- **Attack Pattern:** A higher-level pattern inferred from a Timeline of Alerts.
- **JSON Record:** A stored representation of Parsed Events in structured format.
- **Elasticsearch:** A data store used to index and query JSON Records.
- **Grafana:** A visualization tool used to display system data to users.
- **Dashboard:** The user interface through which users interact with alerts, cases, and system data.
- **System User:** A general user who interacts with the system.
- **SOC Analyst:** A user responsible for investigating alerts and cases.
- **Security Operator:** A user responsible for monitoring system activity.

12. *Alternate Domain Relationships (Logic)*

Core Pipeline Relationships

- PCAP File contains Network Packet
- Network Packet is normalized into Parsed Event
- Parsed Event extracts Observable
- Observable is queried against Enrichment
- Threat Intel Source provides Threat Intel
- Threat Intel supports Enrichment
- Parsed Event is analyzed in Detection Result
- Enrichment is used in Detection Result
- Detection Result indicates Threat
- Detection Result produces Alert

Investigation Relationships

- Case has Timeline
- Timeline orders Alert
- Timeline reveals Attack Pattern
- Dashboard displays Case
- Dashboard displays Alert

Behavior Context Relationships

- Deviation is detected from Baseline Behavior
- Deviation is based on Parsed Event
- Deviation is considered in Detection Result
- Baseline Behavior models Device
- Parsed Event is associated with Device
- Behavior Context feeds Alert

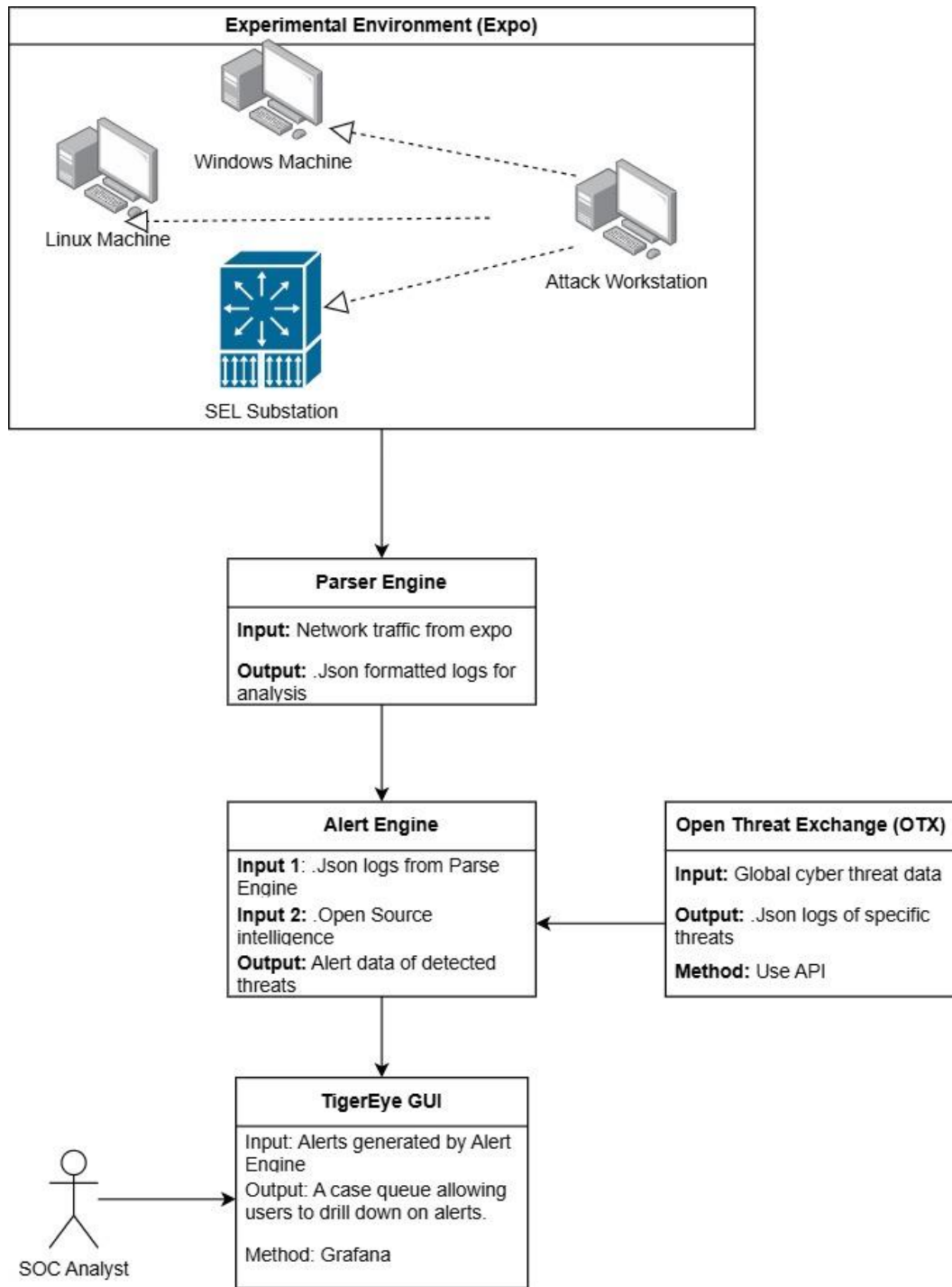
Data & Visualization Relationships

- Parsed Event is stored as JSON Record
- JSON Record is indexed in Elasticsearch
- Elasticsearch is visualized in Grafana
- Grafana feeds Dashboard

Interface Relationships

- **System User uses Dashboard**
- **SOC Analyst investigates with Dashboard**
- **Security Operator monitors with Dashboard**

Conceptual Diagram



6 Risk Mitigation (Aidan)

Risk	Description	Mitigation
Limited tools for interpreting ICS network protocols	Operational technologies tend to have vendor specific net protocols that have limited open-source tools. This could increase our development time overall, due to us having to create specialized classes/functions to parse OT data from scratch.	Our primary mitigation is to use .json as our overall data format. This way we can build a parse engine to take in raw packet capture data, tag interactions with our metadata, and funnel it into our pipeline. Versus, relying on other libraries that might not support industrial protocols.
Lack of reliable OSINT resources with functional APIs	Due to the confidentiality of threats against critical infrastructure, most data sources for traditional artifacts of cyber intelligence (CVEs, IoCs, TTPs) are limited to those with federal connections to CISA. Which creates an intelligence bottle neck for open-source options. Additionally, majority of these threat exchanges do not have federal funding or high commercial investment, so their APIs might not be as refined as mainstream providers such as Mandiant Advantage or Microsoft threat intelligence.	The primary mitigation for under sourced intelligence, is to begin developing your own. From a design perspective, we would shift our focus to verifying data to provide from these sources, instead of using them to inform our platform.
Infrastructure needs for sufficient data pipeline	Our current data pipeline requires an elasticsearch database. This used for long-term storage and to provide information to Grafana for operational dashboards. We currently have this scaffolded	Now that we have the VM up from Cycle 1, next goal is to require sponsorship permission to provide network access to authorized students. That way the sponsor can host the service of the tool, and

	utilizing docker containers, but this has a clear limitation. It can only be done in a snapshot locally, and isn't accessible to the rest of the team. To counteract this Aidan created a VM hosted on McCrary's IT stack, but due to his status as a student, he could not provide network access to all other members.	we as developers can have cross functional access for our data pipeline and graphics.
--	--	---

7 Lessons / Challenges (Amy & Mihir)

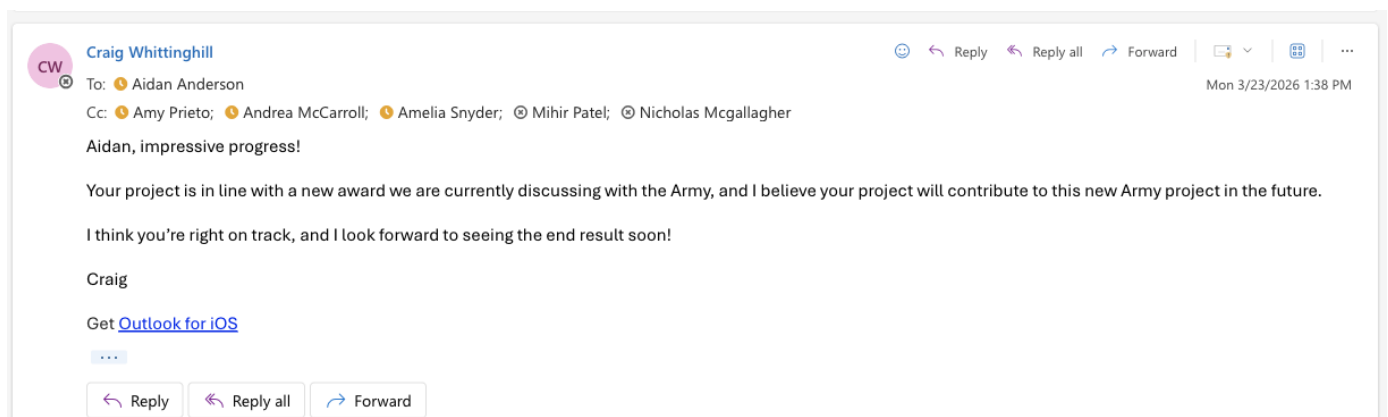
Lessons	Challenges
Arch Spike	
In the arch spike we are learning to navigate new tools like suricata, along with understanding the flow of our team and how we can successfully create our project.	- Understanding best practices to use for our specific mission - Learning new tools, like suricate and Grafana - Researching good open sources for Cyber Intelligence - Coming to a decision on how to format our project
Cycle 1	
We have now begun our cycle For the parser engine, we are learning the basic formatting of .pcap and .json files. Between these two formats, we can read in network data logs and create a database that the alert engine can rely on.	For the parser engines, our challenges were understanding how .pcap data is stored and the best way to extract specific pieces of information, as well as making sure the .pcap files were correctly transformed into .json files.

Logical Flow Questions For the data flow, the team decided to have the parser engine as the first block, pulling raw network traffic, grabbing the relevant information from each packet, and outputting .json files. Then, the alert engine paired with the OTX API would take the formatted .json files and output relevant threat information (severity, suspicious sources/destinations, unusual amounts of traffic, etc.) in .json files. Those threat .json files would be fed into the Elasticsearch database which would then be called by our GUI for a user to analyze.	For the logical flow of data, our challenge came from determining what the best way for data to flow through our product was from a development side and an end user side. It was important to determine this when the product is in its infancy as changing the data flow in later cycles would hinder our future development.
Cycle 2	
Workaround for Database Connectivity Our long-term goal is to have a dedicated virtual machine that is running our elastic search database. This way any member of the team can connect to the virtual machine and work with the database without having it running on their local system. However, network issues have caused issues connecting to the vm remotely. Relying on our sponsor to get the issues fixed for us has caused some slight delays, so a workaround that was created was to have a local docker container running the elastic search database. This way, any member of the team can spin up a container and work with a local instance to test the parser / alert engine code, understand how data is pulled to the Grafana frontend, optimize the database calling process, etc. For now, this is our current setup until the network issues are resolved.	The challenge that caused this lesson was the over-reliance on our sponsor's resources. The virtual machine running the elastic search database is being run with McCrary's resources which involve admin help for network issues or connectivity set up. This over-reliance has caused slight delays during our cycles as we wait on McCrary to come back with a fix, when we could be creating workarounds and continuing with the project. We decided early on, however, to determine a workaround for this issue which is where the idea for docker containers came from. Having the local instances allows each team member to experiment with their own optimizations and understand the database in their own way.

Establishing a joint codebase is critical to development workflows. From Cycle 1 to 2, we all have been working collaboratively via conversations, but from a code perspective it has been siloed. Each developer creates prototypes separately and submits them to the Github. However, in Cycle 3 we will adjust this by making one universal code backend in Python that integrates all engines (Expect GUI of course) to streamline development.	Unifying individual engines into one comprehensive application to increase code fidelity and increase developer visibility.
In Cycle 1, we created a nested file system to store our collected .json files. Throughout Cycle 2, we realized a flat system may work better with our system in the long run as it may be easier to sort through the file system to collect specific data.	Our challenge moving forward is to create a flat file system and stress test it against the nested system within the context of the full engine.

8 Sponsor Approval (Amy)

Cycle 2: We recently conversed with Craig, our team sponsor, via email. Craig informed us that we are on track, and we are falling in line with some other work the institution is doing. We will be in communication with him to see how we can align our project “TigerEye” and project “Army”.



Github: <https://github.com/amy-pri26/TigerEye>